

Full System Design (Updated) — E-Office Productivity Alignment Module

SIH25250 — with Organization layer, Mission concept & Decision-Support Engine

1. The Core Idea — Two Hierarchies Working Together

The system is really built on two parallel hierarchies that mirror each other.

Work hierarchy — who reports to whom, and how work physically flows down:

Government → Department → Organization → Team → Employee → Task

Performance hierarchy — how measured results flow back up:

Task Data → Employee Performance → Team Efficiency → Organization Efficiency → Department Efficiency → Government Intelligence

Work flows down the first ladder. Scores climb back up the second ladder. Everything in this document is either building the first (structure, goals, tasks) or the second (scoring, aggregation, dashboards).

Why an “Organization” layer between Department and Team?
In real government structure, a Department (e.g. Education) contains multiple semi-independent Organizations (e.g. CBSE, Higher Education), and each Organization contains multiple Teams. Going straight from Department to Team skips a real, important layer of accountability — this addition makes the hierarchy match reality more closely.

2. The Actors

Actor	What they do
Super Admin / Government	Sets up departments, organizations, teams; creates high-level missions
Department Head	Allocates a mission’s target across organizations under them
Organization Manager	Divides their target across teams
Team Supervisor	Creates actual tasks, assigns employees, sets deadline & complexity
Employee	Executes tasks, updates status as they work

3. End-to-End Workflow, Phase by Phase

Phase 1 — Government Setup

The Super Admin builds the organizational skeleton once: creates Departments, creates Organizations under each Department, creates Teams under each Organization, creates Employee accounts, and sets reporting relationships.

Example:

```
Government
├── Education Department
│   ├── CBSE Organization
│   │   ├── Examination Team
│   │   └── Administration Team
│   └── Higher Education Organization
│       ├── Scholarship Team
│       └── Admission Team
├── Healthcare Department
│   ├── Hospital Organization
│   └── Health Scheme Organization
└── Police Department
    ├── Cyber Crime Organization
    └── Traffic Organization
```

Phase 2 — Government Creates a Mission

Instead of jumping straight to individual tasks, the government first creates a high-level **Mission** — a large, time-bound target.

Example: *“Process 50,000 pending citizen applications within 7 days.”*

This target cascades down automatically:

```
Government Mission (50,000 applications)
↓
Department Target – Education gets 20,000
↓
Organization Target – Higher Education gets 12,000
↓
Team Target – Scholarship Team gets 3,000
↓
Employee Target – one employee gets 100
```

Phase 3 — Task Distribution

Each level divides its target among the level below it:

- Department splits its target across its Organizations
- Organization splits its target across its Teams
- Team Supervisor creates the actual **Tasks**, assigns them to specific employees, sets a deadline, and tags each task’s complexity (Low/Medium/High)

Phase 4 — Employee Executes Work

The employee sees their assigned tasks on a simple dashboard and updates status as they progress:

```
Pending → In Progress → Waiting for External Department → Completed
                                     → (or) Rejected/Rework
```

Every status change is automatically timestamped. This timestamped history is the raw data the entire scoring system depends on — nothing is manually scored by a person.

Phase 5 — The Scoring Brain

A separate backend service reads each task’s status history, deadline, completion time, complexity, and quality outcome, and produces four sub-scores, combined into one final score.

Worked example:

Component	Score
Volume	85
Timeliness	92
Quality	95
Complexity	88

With role weights Volume 25%, Timeliness 30%, Quality 30%, Complexity 15%:

```
Final Score = (85×0.25) + (92×0.30) + (95×0.30) + (88×0.15) = 91.25
```

Weights are configurable per role — never hardcoded.

Phase 6 — Team Efficiency

A team's score is **not** a simple average of its employees' scores. The aggregation engine also factors in completed workload, complexity mix, timeliness, quality, number of employees, and task type — so a team that did more, harder work isn't penalized for looking "average" next to a smaller team that only did easy tasks.

Phase 7 — Organization Efficiency

Team scores roll up into an Organization score the same way — weighted, not simply averaged. The Organization dashboard can show overall efficiency, trend over time, a ranked list of its teams, and its strongest/weakest scoring dimension (e.g. "Best: Timeliness 95%, Weak: Quality 84%").

Phase 8 — Department Efficiency

Organization scores roll up into a Department score. A department head can immediately see which of their organizations is outperforming another, and drill down to see why.

Phase 9 — Government Intelligence Dashboard

At the top, all department scores roll up into a single government-wide view. A senior official can ask "which department is performing best right now?", see the answer, then drill down further — "which organization?", "which team?" — all the way to a specific, provably best-performing team.

This is the point where the platform stops being a productivity tracker and becomes a decision-support system.

4. Why We Never Naively Average Scores Upward

A dangerous shortcut would be: `Government Score = average(all employee scores)`. This is misleading — it would let a 5-person team have the same influence on the government score as a 500-person organization.

Instead, every level up the aggregation ladder is a **weighted** combination, where the weight reflects real workload and complexity handled at that level:

Task-level metrics → Employee score → Weighted Team score
→ Weighted Organization score → Weighted Department score → Government score

5. The Decision-Support Engine (The Standout Feature)

This is what turns the platform from “who is productive?” into **“based on historical evidence, who should handle this urgent work?”**

When the government creates an urgent task category (e.g. “Emergency document verification”), the system:

Task Category → Pull historical tasks of this type → Filter relevant teams
→ Calculate their recent performance → Rank teams → Return recommendation

Example output:

Rank	Team	Score	Similar tasks handled	On-time %	Quality %
1	Digital Verification Team	96%	143	97%	95%
2	Document Processing Team	92%	—	—	—
3	Admin Verification Team	87%	—	—	—

This should be the centerpiece of a hackathon demo — it’s the feature that proves the platform gives government officials real, evidence-based decisions, not just a report card.

6. The Five Dashboards

Rather than building many disconnected screens, the platform is organized into five layered dashboard experiences, each drilling into the next:

1. Government / Super Official — overall efficiency, department ranking, performance trend, active missions, delayed work, top-performing organizations, and the “Recommend Team” decision-support tool.

2. Department Head — department efficiency broken into Quality/Timeliness/Complexity, ranked list of organizations underneath, and drill-down into any one of them.

3. Organization Manager — organization score, active/completed/delayed task counts, ranked list of teams underneath.

4. Team Supervisor — operational view: today’s pending/in-progress/waiting/completed/rejected counts, a table of team members with their score, task count, and quality %, plus tools to create tasks, assign work, and redistribute if someone is overloaded.

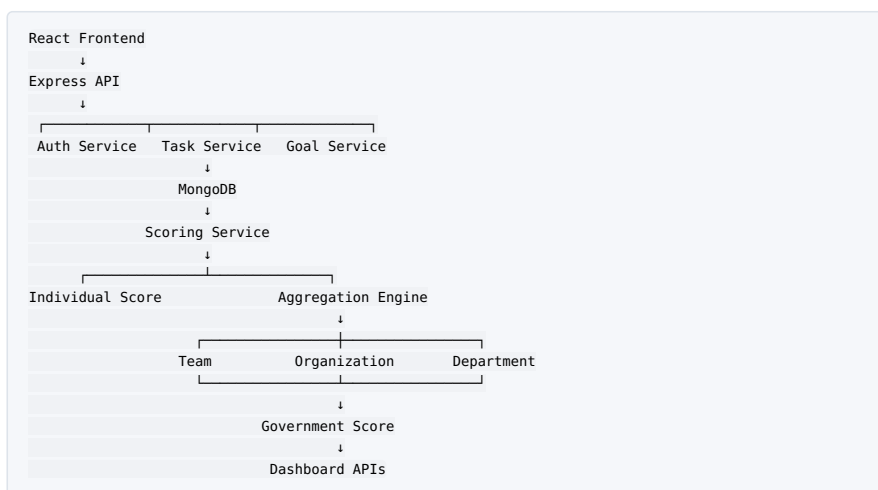
5. Employee — their own overall score, the four sub-scores (Volume/Timeliness/Quality/Complexity), their task list, and — most importantly — a “Why is my score X?” breakdown showing exactly which tasks and numbers produced it.

7. Database Design (MongoDB Collections)

Collection	What it stores
User	Name, employeeId, role, roleType, department/organization/team links, reportingTo
Department	Name, code, head, description
Organization	Name, code, linked Department, head
Team	Name, code, linked Department + Organization, supervisor
Mission	Government-level target, deadline, priority, and per-department target split
Goal	A measurable target at any level (government/department/organization/team/individual), linked to its parent goal and its Mission
Task	Title, linked Goal/Mission, assignedTo, complexity, priority, deadline, status, statusLog
TaskStatusLog	Embedded inside Task — every status change with a timestamp and who changed it
RoleWeightConfig	Per-role weight settings for Volume/Timeliness/Quality/Complexity, with effective date ranges
ScoreSnapshot	A permanent historical record — never overwritten — storing every score ever calculated, at every entity level (employee/team/organization/department/government)

This structure directly supports both hierarchies: the Department → Organization → Team → User chain (work hierarchy), and Task → ScoreSnapshot at every level (performance hierarchy).

8. Backend Architecture



The scoring logic lives in its own service folder, not inside route controllers, so it stays testable and independent of the API layer:

```

/server/services/scoring/
  volume.service.js
  timeliness.service.js
  quality.service.js
  complexity.service.js
  finalScore.service.js
  aggregation.service.js

```

9. Tech Stack

Layer	Technology
Frontend	React (Vite) + Tailwind CSS
Charts	Recharts
Backend	Node.js + Express.js
Database	MongoDB (Mongoose)
Scheduled jobs	node-cron
Auth	JWT, role-based (Government/Department/Organization/Team/Employee)

No ML, no AI agents anywhere in the scoring logic — every score stays mathematically explainable and auditable, which matters when scores may feed into real appraisals.

10. Build Order (Matched to MERN Skills)

Phase 1 — Foundation: React + Vite + Tailwind, Node + Express, MongoDB + Mongoose, JWT auth.

Phase 2 — Organization hierarchy: Department, Organization, Team, and User CRUD — get Department → Organization → Team → Employee working correctly first.

Phase 3 — Mission + Goal: Build Mission creation and the Goal cascade (Government → Department → Organization → Team → Employee targets).

Phase 4 — Task management: Create/assign tasks, deadlines, priority, complexity, status, and status history logging.

Phase 5 — Scoring brain: Volume, Timeliness, Quality, Complexity functions — pure math, no AI.

Phase 6 — Aggregation: Employee → Team → Organization → Department → Government, using weighted (not naive average) combination.

Phase 7 — Dashboards: Build in dependency order — Employee first, then Team, Organization, Department, Government — since each higher dashboard depends on data from the level below.

Phase 8 — Decision support: Urgent mission → task type → historical performance → ranked recommendation. This is your final demo moment.

11. Suggested Hackathon MVP Scope

Don't attempt every government department — build one clean, realistic vertical slice:

```

Government
├── Education
│   ├── Scholarship Organization
│   │   ├── Verification Team
│   │   └── Processing Team
│   └── Healthcare
│       ├── Health Scheme Organization
│       │   ├── Verification Team
│       │   └── Claims Team

```

Seed roughly: 2 Departments, 4 Organizations, 8 Teams, 30 Employees, 200–300 Tasks.

Demo story: Government creates an urgent mission → Education receives its target → Organization divides it → Team receives tasks → Employees work and update status → system logs timestamps → scoring engine calculates the four metrics → scores roll up through Team → Organization → Department → Government dashboard → official asks “which team should handle the next urgent mission?” → system recommends the best-performing team, backed by historical evidence.

This tells one clear, complete story: task creation → real work → measurable performance → government-level decision — while keeping every score fully explainable, fair (external delays excluded), and free of AI/ML.